
Session 04 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Lambda Functions

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py
Enter a number: 5
Square using lambda function: 25
Square using normal function: 25
PS C:\Users\rathi_pu888d0\Desktop\ITR> |
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

Q1. (Lambda Function)

Write a lambda function named square that takes a number x and returns its square.

Call the lambda function with input from the user and print the result.

Also write the same logic using a normal def function and compare both in comments.

```
'''
```

```
# Lambda function to calculate square
```

```
square = lambda x: x ** 2
```

```
# Normal function to calculate square
```

```
def square_def(x):
```

```
    return x ** 2
```

```
# Take input from the user
```

```
num = int(input("Enter a number: "))
```

```
# Call the lambda function
```

```
print("Square using lambda function:", square(num))
```

```
# Call the normal function
```

```
print("Square using normal function:", square_def(num))
```

```
"""Explanation in Comments"""
```

```
# A lambda function is a small anonymous function written in a single line.
```

```
# It is useful for simple operations and does not require the def keyword.
```

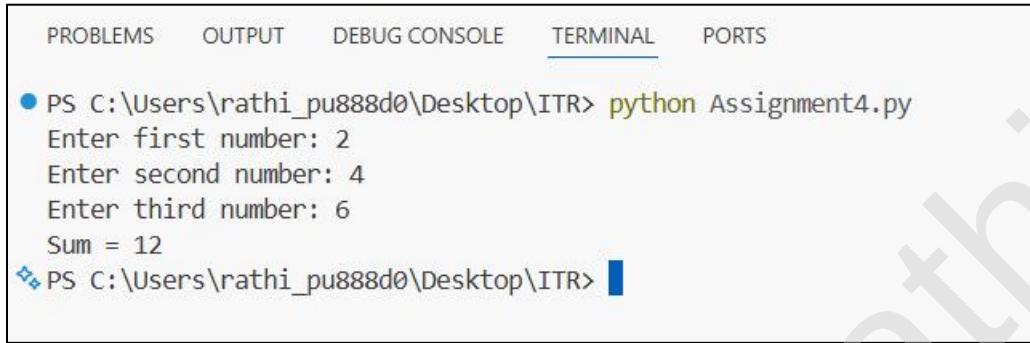
```
# A normal function is created using the def keyword and can contain multiple statements.
```

```
# Lambda functions are shorter, while normal functions are more readable for complex tasks.
```

```
# Both functions perform the same operation of calculating the square of a number.
```

Q2. Lambda with Multiple Arguments

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py
Enter first number: 2
Enter second number: 4
Enter third number: 6
Sum = 12
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

""" ASSIGNMENT 4 """

'''

Q2. (Lambda with Multiple Arguments)

Create a lambda function `add_three(a, b, c)` that returns the sum of three numbers.

Take three numbers as input from the user and print their sum using this lambda function.

'''

Lambda function to calculate the sum of three numbers

`add_three = lambda a, b, c: a + b + c`

Take three numbers as input from the user

`num1 = int(input("Enter first number: "))`

`num2 = int(input("Enter second number: "))`

`num3 = int(input("Enter third number: "))`

Print the sum

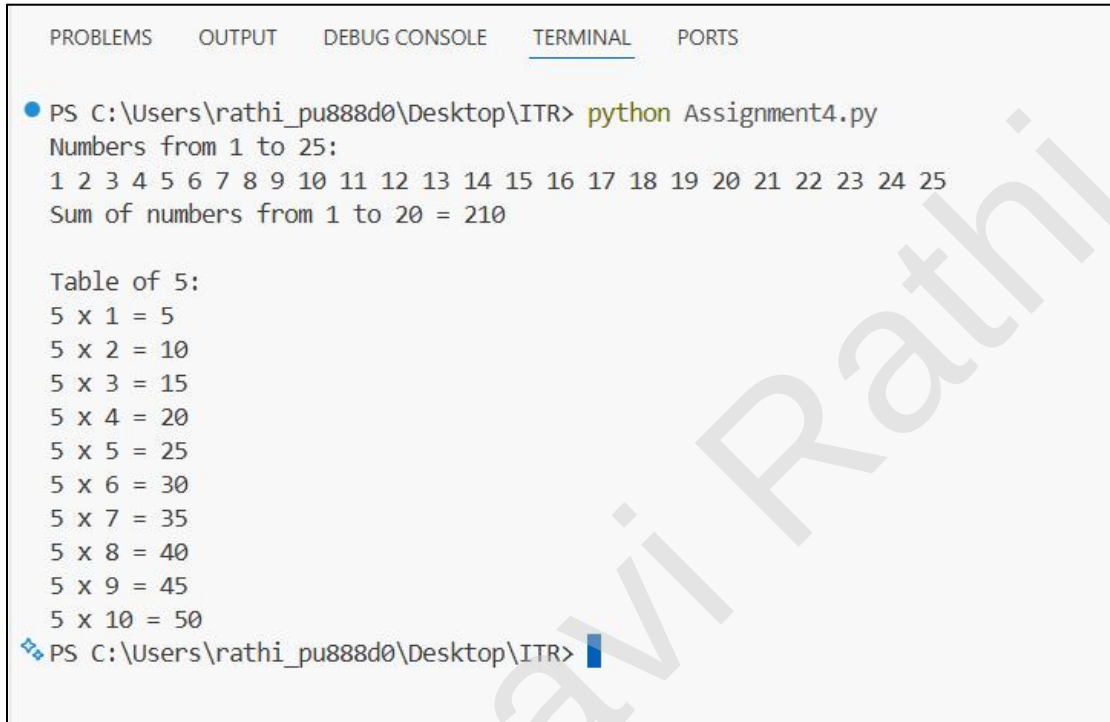
`print("Sum =", add_three(num1, num2, num3))`

"Explanation in Comments"

```
# The lambda function add_three() takes three arguments.  
# It returns the sum of the three numbers.  
# The user enters three numbers.  
# These numbers are passed to the lambda function.  
# The returned sum is then displayed.
```

Q3. For Loop Revision

Output Screenshot:



```
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py
Numbers from 1 to 25:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
Sum of numbers from 1 to 20 = 210

Table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

Q3. (For Loop Revision)

Write a program using a for loop and range() to:

- a) Print all numbers from 1 to 25.
- b) Print the sum of all numbers from 1 to 20.
- c) Print the table of 5 (5×1 to 5×10).

Add comments explaining when to use for loop.

```
'''
```

a) Print all numbers from 1 to 25

```
print("Numbers from 1 to 25:")
```

```
for i in range(1, 26):
```

```
    print(i, end=" ")
```

```
print()
```

b) Print the sum of all numbers from 1 to 20

```
total = 0
```

```
for i in range(1, 21):
```

```
    total += i
```

```
print("Sum of numbers from 1 to 20 =", total)
```

```
print()
```

c) Print the table of 5

```
print("Table of 5:")
```

```
for i in range(1, 11):
```

```
    print("5 x", i, "=", 5 * i)
```

"""Explanation in Comments"""

A for loop is used when the number of iterations is known.

range() generates a sequence of numbers for the loop.

range(1, 26) generates numbers from 1 to 25.

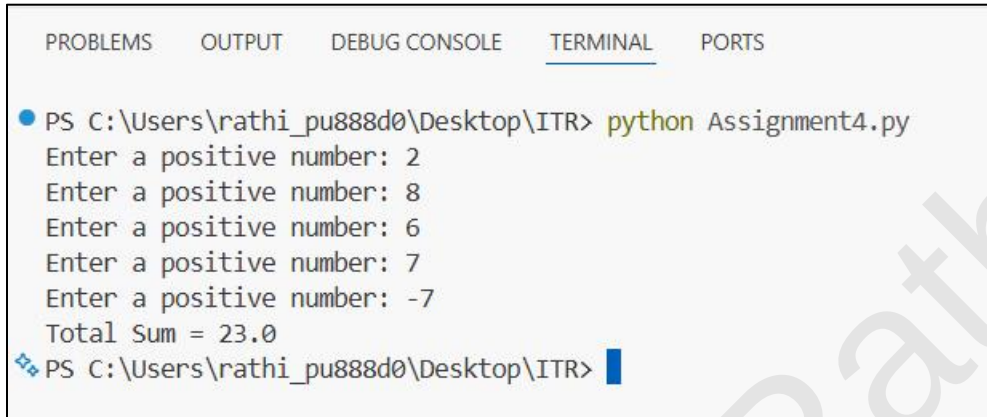
range(1, 21) generates numbers from 1 to 20 for calculating the sum.

range(1, 11) generates numbers from 1 to 10 for the multiplication table.

for loops make it easy to repeat a task a fixed number of times.

Q4. While Loop Revision

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment4.py
Enter a positive number: 2
Enter a positive number: 8
Enter a positive number: 6
Enter a positive number: 7
Enter a positive number: -7
Total Sum = 23.0
PS C:\Users\rathi_pu88d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

Q4. (While Loop Revision)

Write a program using a while loop that:

Repeatedly asks the user to enter a positive number.

Keeps adding the numbers to a running total.

Stops when the user enters 0 or a negative number.

Finally prints the total sum.

```
'''
```

```
# Initialize the total sum
```

```
total = 0
```

```
# Take input from the user
```

```
number = float(input("Enter a positive number: "))
```

```
# Continue while the entered number is positive
```

```
while number > 0:
```

```
    # Add the number to the running total
```

```
    total += number
```

```
    # Ask for the next number
```

```
    number = float(input("Enter a positive number: "))
```

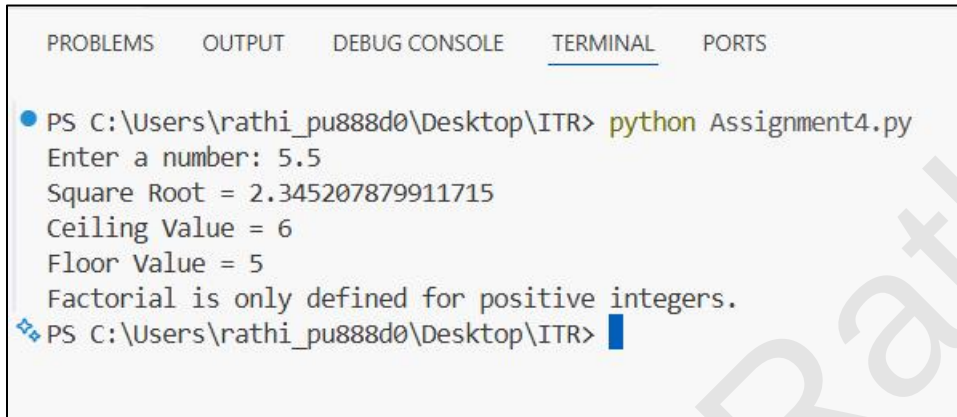
```
# Display the final sum  
print("Total Sum =", total)
```

'''Explanation in Comments'''

```
# The variable 'total' stores the running sum of all positive numbers.  
# The while loop continues as long as the user enters a positive number.  
# Each entered number is added to the total.  
# The loop stops when the user enters 0 or a negative number.  
# Finally, the program prints the total sum.
```

Q5. Math Module

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py
Enter a number: 5.5
Square Root = 2.345207879911715
Ceiling Value = 6
Floor Value = 5
Factorial is only defined for positive integers.
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

```
Q5. (Math Module)
```

Write a program that imports the math module and does the following:

Takes a number as input.

Prints its square root using `math.sqrt()`.

Prints its factorial using `math.factorial()` (for positive integers only).

Prints the ceiling and floor value using `math.ceil()` and `math.floor()`.

Add comments about the usefulness of the math module.

```
'''
```

```
# Import the math module
```

```
import math
```

```
# Take input from the user
```

```
num = float(input("Enter a number: "))
```

```
# Print square root
```

```
print("Square Root =", math.sqrt(num))
```

```
# Print ceiling and floor values
```

```
print("Ceiling Value =", math.ceil(num))
```

```
print("Floor Value =", math.floor(num))
```

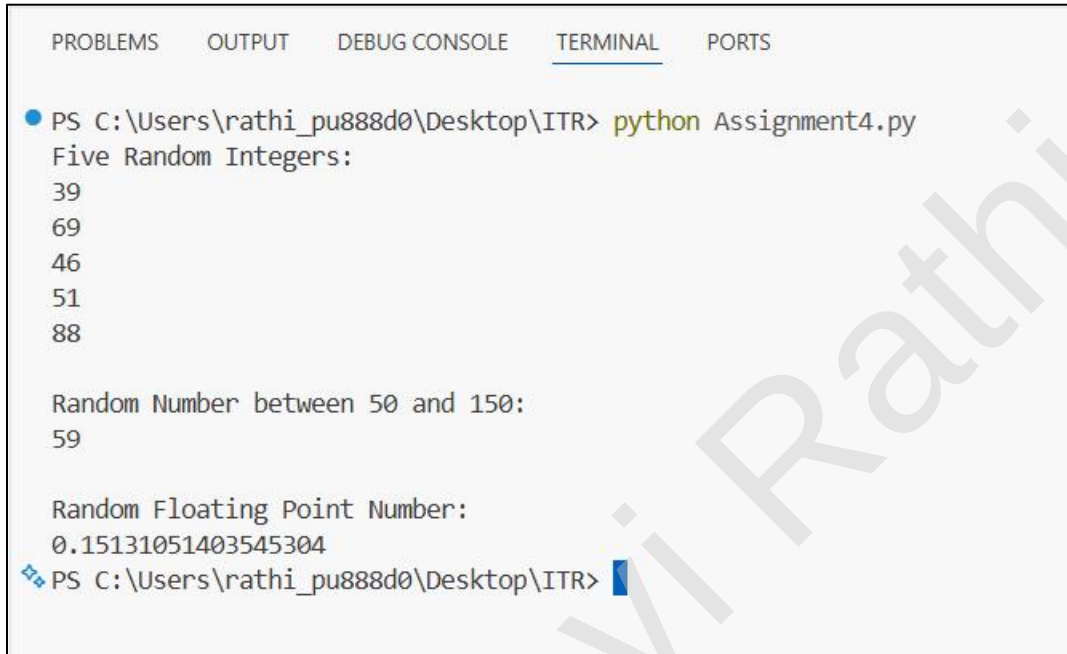
```
# Print factorial only for positive integers
if num >= 0 and num == int(num):
    print("Factorial =", math.factorial(int(num)))
else:
    print("Factorial is only defined for positive integers.")
```

'''Explanation in Comments'''

```
# The math module provides many useful mathematical functions.
# math.sqrt() is used to calculate the square root of a number.
# math.factorial() is used to calculate the factorial of a positive integer.
# math.ceil() returns the smallest integer greater than or equal to the number.
# math.floor() returns the largest integer less than or equal to the number.
# Using the math module makes mathematical calculations easier and more efficient.
```

Q6. Random Module

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment4.py
Five Random Integers:
39
69
46
51
88

Random Number between 50 and 150:
59

Random Floating Point Number:
0.15131051403545304
PS C:\Users\rathi_pu88d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

```
Q6. (Random Module)
```

```
Write a program using the random module that:
```

```
Generates and prints 5 random integers between 1 and 100.
```

```
Generates and prints one random number between 50 and 150.
```

```
Prints a random floating point number between 0 and 1.
```

```
Use different functions from the random module.
```

```
'''
```

```
# Import the random module
```

```
import random
```

```
# Generate and print 5 random integers between 1 and 100
```

```
print("Five Random Integers:")
```

```
for i in range(5):
```

```
    print(random.randint(1, 100))
```

```
# Generate and print one random integer between 50 and 150
print("\nRandom Number between 50 and 150:")
print(random.randrange(50, 151))
```

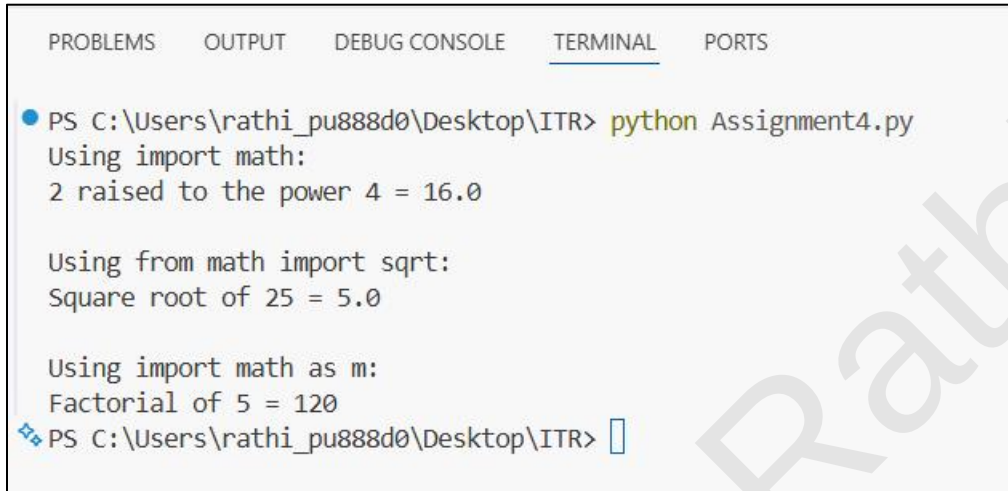
```
# Generate and print a random floating-point number between 0 and 1
print("\nRandom Floating Point Number:")
print(random.random())
```

"Explanation in Comments"

```
# The random module is used to generate random values.
# random.randint(a, b) generates a random integer between a and b (inclusive).
# random.randrange(start, stop) generates a random integer within the given range.
# random.random() generates a random floating-point number between 0 and 1.
# Random numbers are commonly used in games, simulations, and testing.
```

Q7. Import Methods

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py
Using import math:
2 raised to the power 4 = 16.0

Using from math import sqrt:
Square root of 25 = 5.0

Using import math as m:
Factorial of 5 = 120
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

```
Q7. (Import Methods)
```

Demonstrate different ways of importing modules:

1. Import the entire math module using import math and use math.pow(2, 4).
2. Import only sqrt from math using from math import sqrt.
3. Import math with an alias as m and use m.factorial(5).

Print the results with clear messages.

```
'''
```

```
# Import the entire math module
import math
```

```
print("Using import math:")
print("2 raised to the power 4 =", math.pow(2, 4))
```

```
# Import only sqrt from the math module
from math import sqrt
```

```
print("\nUsing from math import sqrt:")
print("Square root of 25 =", sqrt(25))
```

Import math module with an alias

import math as m

print("\nUsing import math as m:")

print("Factorial of 5 =", m.factorial(5))

"Explanation in Comments"

import math imports the entire math module.

Functions are accessed using the module name, such as math.pow().

from math import sqrt imports only the sqrt() function.

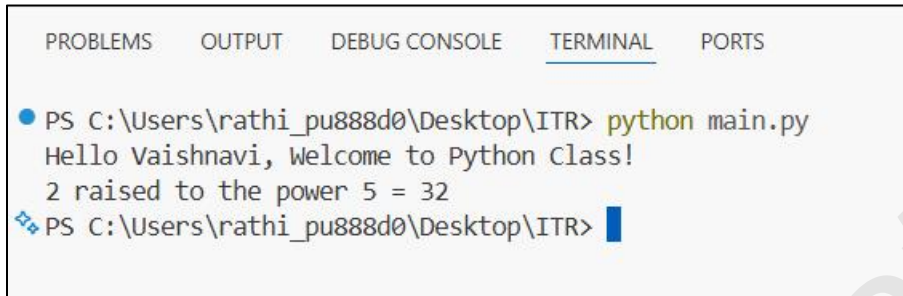
The sqrt() function can be used directly without writing math.sqrt().

import math as m creates an alias (short name) for the module.

Aliases make code shorter and easier to write.

Q8. Custom Module

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python main.py
Hello Vaishnavi, Welcome to Python Class!
2 raised to the power 5 = 32
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
"""
Custom Module: mymodule.py
"""

# Function to display a greeting message
def greet(name):
    print("Hello", name + ", Welcome to Python Class!")

# Function to calculate power
def calculate_power(base, exp):
    return base ** exp
```

```
""" ASSIGNMENT 4 """
```

```
"""
```

Q8. (Custom Module)

Create a custom module file named mymodule.py that contains:

A function greet(name) that prints
"Hello [name], Welcome to Python Class!"

A function calculate_power(base, exp) that returns
base raised to exponent.

Then write a main program that imports this module and uses both functions.

'''

Import the custom module

import mymodule

Call the greet() function

mymodule.greet("Vaishnavi")

Call the calculate_power() function

result = mymodule.calculate_power(2, 5)

print("2 raised to the power 5 =", result)

'''Explanation in Comments'''

A custom module is a Python file containing functions or variables.

The module is imported using the import keyword.

greet() displays a welcome message.

calculate_power() returns the value of base raised to exponent.

Using modules helps organize code and promotes reusability.

Q9.name == "main"

Output Screenshot:



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. It shows a command prompt where the user has run 'python Assignment4.py'. The output of the script is displayed: 'Welcome to Python Programming!' followed by 'This program is being executed directly.' on the next line. The prompt is now 'PS C:\Users\rathi_pu888d0\Desktop\ITR>'.

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

```
Q9. (__name__ == "__main__")
```

Write a program that:

a) Creates a small function and demonstrates the use of
if __name__ == "__main__":

so that some code runs only when the file is executed directly.
'''

```
# Function definition
```

```
def greet():  
    print("Welcome to Python Programming!")
```

```
# This block runs only when the file is executed directly
```

```
if __name__ == "__main__":  
    greet()  
    print("This program is being executed directly.")
```

"Explanation in Comments"

```
# The greet() function displays a welcome message.  
# __name__ is a special built-in variable in Python.  
# When a file is run directly, __name__ is set to "__main__".  
# The code inside if __name__ == "__main__": executes only when the file is run  
directly.  
# If the file is imported as a module, this block does not execute.
```

Vaishnavi Rathni

Q10. Mini Project - Combined Concepts

Output Screenshot:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment4.py

===== Math Utility Program =====
1. Square
2. Power
3. Random Number
4. Exit
Enter your choice: 1
Enter a number: 4
Square = 16.0

===== Math Utility Program =====
1. Square
2. Power
3. Random Number
4. Exit
Enter your choice: 2
Enter base: 2
Enter exponent: 4
Power = 16.0

===== Math Utility Program =====
1. Square
2. Power
3. Random Number
4. Exit
Enter your choice: 3
Random Number = 82

===== Math Utility Program =====
1. Square
2. Power
3. Random Number
4. Exit
Enter your choice: 4
Exiting Program...
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 4 """
```

```
'''
```

Q10. (Mini Project - Combined Concepts)

Create a Simple Math Utility Program using functions, lambda, loops and modules.

Write the following:

1. A lambda function to calculate square of a number.
2. A normal function using math module to calculate power.

3. Use a while loop to repeatedly ask the user for a number and choice (Square / Power / Random Number).
4. Use random module to generate a random number when chosen.

Continue until the user chooses to exit.

Use proper functions and comments.

```
"""

# Import required modules
import math
import random

# Lambda function to calculate square
square = lambda x: x ** 2

# Function to calculate power using math module
def calculate_power(base, exponent):
    return math.pow(base, exponent)

# Main program using while loop
while True:

    print("\n===== Math Utility Program =====")
    print("1. Square")
    print("2. Power")
    print("3. Random Number")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        number = float(input("Enter a number: "))
        print("Square =", square(number))

    elif choice == 2:

        base = float(input("Enter base: "))
        exponent = float(input("Enter exponent: "))

        print("Power =", calculate_power(base, exponent))

    elif choice == 3:

        print("Random Number =", random.randint(1, 100))
```

```
elif choice == 4:

    print("Exiting Program...")
    break

else:

    print("Invalid Choice. Please try again.")
```

"""Explanation in Comments"""

```
# The math module is used for mathematical calculations.
# The random module is used to generate random numbers.
# The lambda function calculates the square of a number.
# The calculate_power() function uses math.pow() to calculate powers.
# The while loop repeatedly displays the menu until the user chooses Exit.
# if-elif-else statements are used to perform different operations.
# random.randint(1, 100) generates a random integer between 1 and 100.
```
